Полиморфизм

Подтемы занятия

- проблема дублирования кода
- понятие полиморфизма
- виртуальные методы
- позднее связывание
- абстрактные классы
- срезка объектов

Проблема дублирования кода

Задача: Написать функцию ***run_test_query*** для выполнения запроса к разным БД (MSSQL, PostgreSQL)

PostgresConnection
• connect(string connection_string) • execute_query(string query) • disconnect()

MssqlConnection
• connect(string connection_string) • execute_query(string query) • disconnect()

Наивное решение – для каждого типа подключения реализовать собственную версию ***run_test_query*** (перегрузить)

1. *void run_test_query(PostgresConnection& conn) { ... }*
2. *void run_test_query(MssqlConnection& conn) { ... }*

Недостатки перегрузки функции

Дублирование кода

Тела функций практически одинаковы. Изменение логики работы функции потребует правок во все версии

Сложность расширения

- Добавили Oracle? Пишем третью перегрузку.
- Добавили MySQL? Пишем четвёртую.

Жёсткая привязка

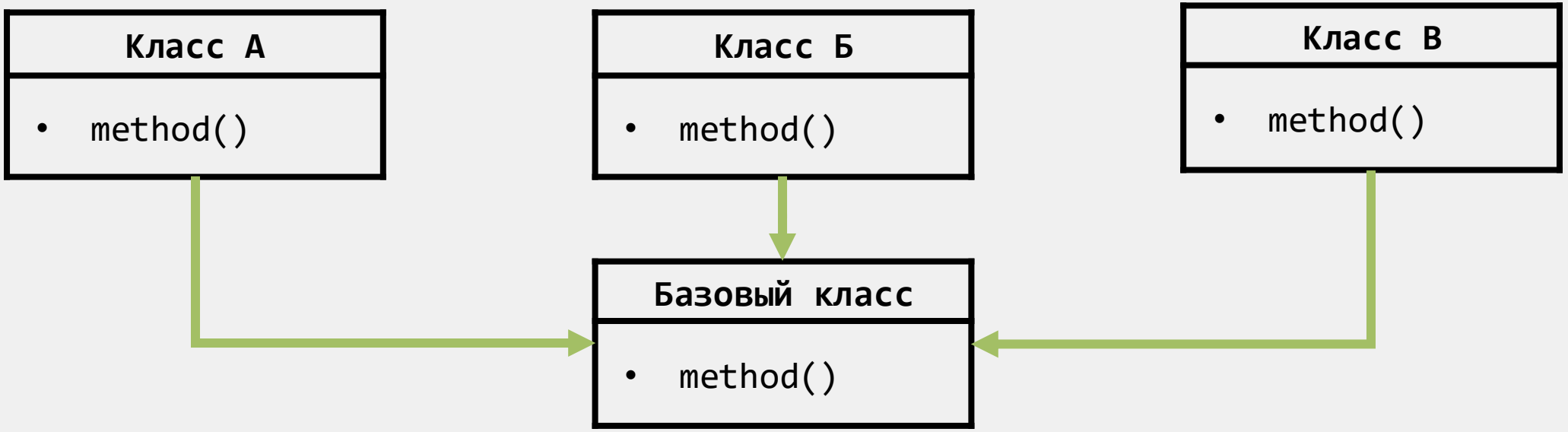
Вызывающий код обязан знать конкретный тип объекта и явно выбирать нужную функцию.

Для того, чтобы избежать данные проблемы, реализации подключений необходимо объединить под ***единый*** интерфейс.

Понятие полиморфизма

Полиморфизм – принцип ООП, позволяющий объектам с одинаковым интерфейсом иметь различную реализацию поведения.

Как это работает?



Код вызывает *method()* через базовый класс, а во время исполнения программы выбирается ***реальная*** реализация объекта.

Применение в разработке

	Применение	*Базовый класс*	*Интерфейсные методы*	*Преимущество интерфейса*
	Провайдеры БД	DbConnection	• connect() • executeQuery() • disconnect()	Прикладной код не зависит от конкретной СУБД
	Графические редакторы	Shape	• draw() • resize() • fill_color()	Отрисовка всех фигур на полотне в одном цикле
	Игровые движки	GameObject	• render() • relocate() • destroy()	Обход всех объектов сцены без проверки типов

Виртуальные методы

Обычный метод

Использует ***раннее*** связывание (на этапе компиляции)

Virtual метод

Использует ***позднее*** связывание (во время работы программы)

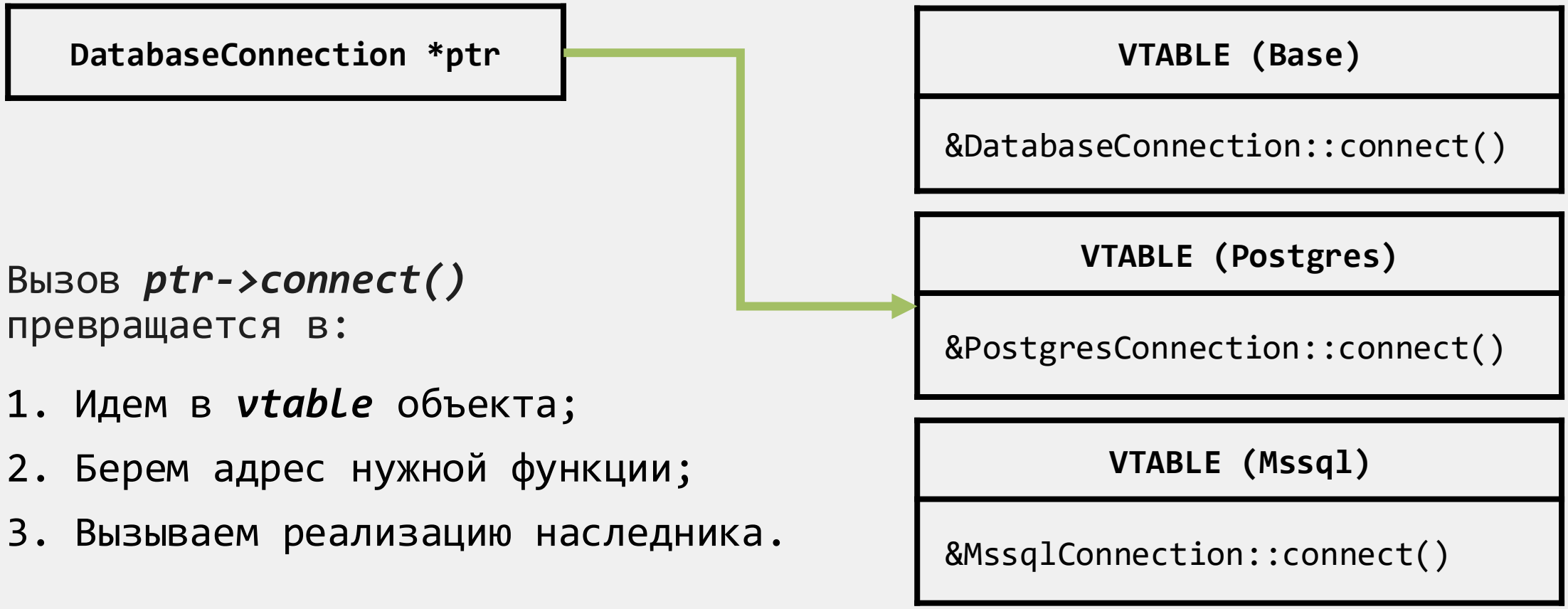
```
class DatabaseConnection {
   public:
      virtual void connect(const string& conn_str) {
         // Базовая реализация.
      }
};
```

Ключевое слово: **virtual**

Заставляет компилятор создать таблицу виртуальных функций (***vtable***)

Таблица vtable

Как компилятор понимает, какой метод вызвать?



Вызов ***ptr->connect()*** превращается в:

1. Идем в ***vtable*** объекта;
2. Берем адрес нужной функции;
3. Вызываем реализацию наследника.

Абстрактные классы

Что если базовому классу НЕЧЕГО реализовать?
(например, общий запрос к БД не имеет смысла)

В C++ используются чисто виртуальные функции:

```
virtual std::string execute_query(const std::string& sql) = 0;
```

Класс с хотя бы одной такой функцией становится ***абстрактным***.

Нельзя создать экземпляр абстрактного класса: только ссылку или указатель на него

Наследник обязан реализовать все чисто виртуальные методы, иначе он тоже станет абстрактным.

Пример полиморфизма

Единая функция для всех баз данных:

```
void run_query(DatabaseConnection& db, const std::string& connection_str, const std::string& sql) {
    db.connect(connection_str);
    std::string result = db.execute_query(sql);
    std::cout << "Получен результат: " << result << std::endl;
    db.disconnect();
}
```

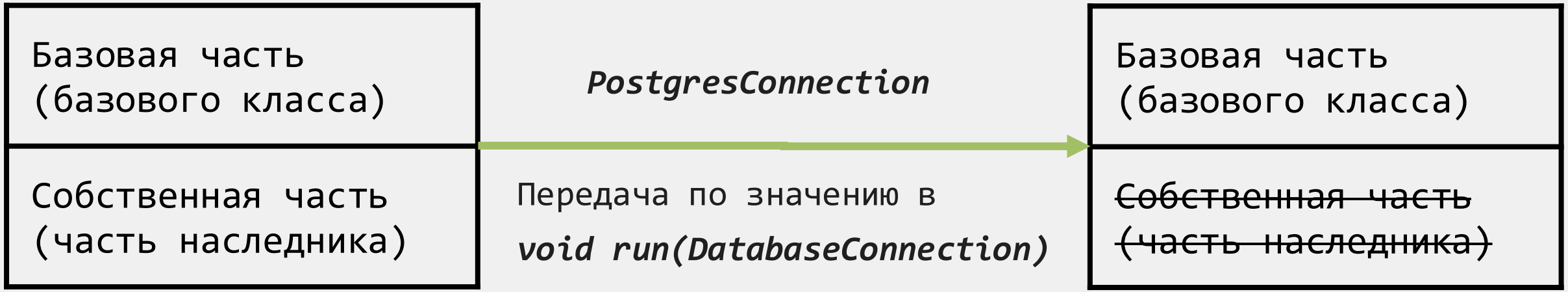
Вызов:

```
PostgresConnection pg;
MssqlConnection ms;

run_query(pg, ...); // Неявно вызовутся методы PostgreSQL
run_query(ms, ...); // Неявно вызовутся методы MSSQL
```

Срезка объектов

Что будет при передаче по значению?



Специфическая часть для ***PostgresConnection*** будет уничтожена.

В функции будут вызваны методы базового класса, а не наследника.

Конец презентации

Разработчик

Марышев А.В.

Контакты для связи

Телеграм: @XekS_s